



Mobile Application Security

- Lecture By
Pratidnya S. Hegde Patil
Assistant Professor-IT Dept.



Topics

- Mobile Attack
- Mobile Attack Vector and Vulnerability
- OWASP Mobile Top10 Testing
- Mobile Attack Countermeasures

Courtesy: Ethical Hacking Essentials
Documentation by EC council



Mobile Attack

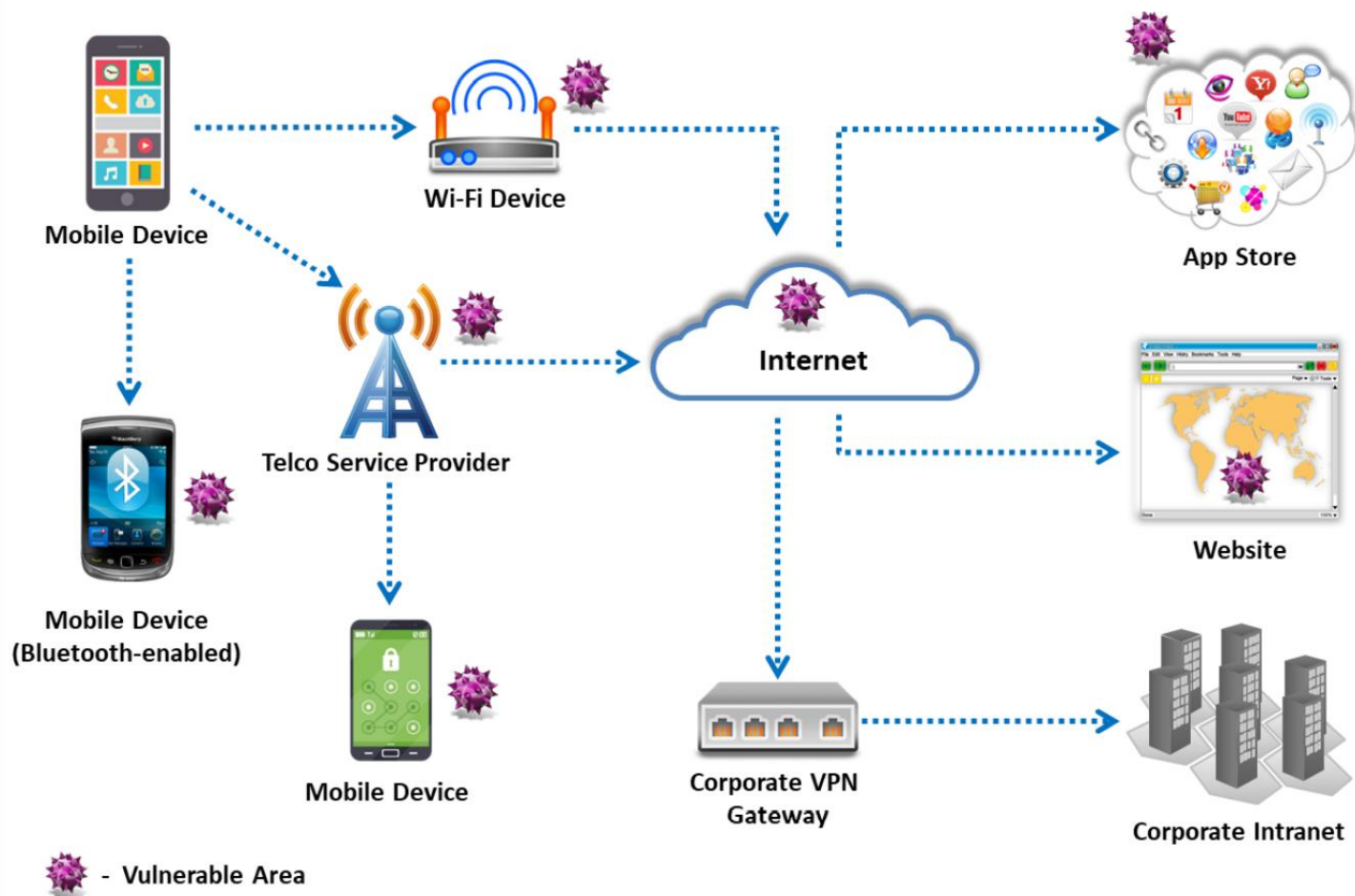
- Mobility has become the key parameter for internet usage.
- Mobiles are replacing desktops and laptops as they allow users to
 - Access internet
 - Email
 - GPS navigation
 - Critical data such as – contact lists, passwords, calendars and login credentials
 - Perform online transactions – including purchase of goods and applications over wireless networks, redemption of coupons and tickets, and banking from their smartphones.



Mobile Attack

- Security threats **exploit critical data** as well as **financial information** and other details of mobile users and may also **damage the reputation of mobile networks and organizations.**

Vulnerable Areas in Mobile Business Environment



OWASP Mobile Top 10 Testing (2016)

<https://owasp.org/www-project-mobile-top-10/#>

- M1: Improper Platform Usage
- M2: Insecure Data Storage
- M3: Insecure Communication
- M4: Insecure Authentication
- M5: Insufficient Cryptography
- M6: Insecure Authorization
- M7: Client Code Quality
- M8: Code Tampering
- M9: Reverse Engineering
- M10: Extraneous Functionality



M1: Improper Platform Usage

- This category covers the misuse of a platform feature or the failure to use platform security controls. It includes Android intents, platform permissions, and the misuse of Touch ID, Keychain, or some other security control that is part of the mobile device's OS. There are several ways in which mobile apps can be exposed to this risk.



M1: Improper Platform Usage

■ Real-World Case:

- Three iOS apps: “Fitness Balance app”, “Heart Rate Monitor”, and “Calories Tracker app” came into light for bypassing Apple’s Touch ID. They were asking users to use their fingerprint to get fitness information, while they were using it to charge money from the App Store.



M1: Improper Platform Usage

Misuse of Platforms

Misusing the Android or iOS platform is the greatest security vulnerability for mobile devices. Apps can unintentionally misuse platforms.

To address this vulnerability:

- Follow the platform's security guidelines.
- Utilize the server-side security features available.
- Use secure configurations and coding practices.
- Harden the server-side.
- Limit data transmission between applications.
- Limit file access permissions.
- Encrypted stored data.



M2: Insecure Data Storage

- Insecure data storage vulnerability arises when development teams assume that users and malware will not have access to a mobile device's file system and subsequently to sensitive information in the device's data stores. “Jailbreaking” or rooting a mobile device bypasses encryption protection mechanisms. OWASP recommends analyzing platforms' data security application programming interfaces (APIs) and calling them appropriately.
- Unintended data leakage occurs when a developer unintentionally places sensitive data in a location on the mobile device that is easily accessible by other apps on the device. Such leakage is normally caused by vulnerabilities in the OS, frameworks, compiler environment, new hardware, and so on without a developer's knowledge. It is a significant threat to the OS, platforms, and frameworks; thus, it is important to understand how they handle features such as URL caching, browser cookie objects, and HTML5 data storage.



M2: Insecure Data Storage

■ Real-World Case:

- Dating apps like Tinder, OKCupid, and Bumble have time and again been scrutinized for their insecure data storage practices. The security lapses present on these apps vary according to feasibility and severity and feasibility, can expose users' name, login details, message history, and even location, in addition to other personal account activity.



M2: Insecure Data Storage

Insecurely Stored Data

Insecure data storage allows attackers to exploit stolen mobile devices and steal sensitive information. Sometimes, it is unavoidable to have applications store sensitive data, but it is essential to ensure this data is not accessible to unauthorized users. Encryption alone is not always enough.

To secure stored data:

- Encrypt all data.
- Use authorization and authentication mechanisms for accessing mobile apps.
- Restrict mobile app access to stored files.
- Use secure coding practices.



M3: Insecure Communication

- This category covers poor handshaking, incorrect SSL versions, weak negotiation, clear text communication of sensitive assets, and so on. Such flaws expose an individual user's data and can lead to account theft.
- If the adversary intercepts an admin account, the entire site could be exposed.
- A poor Secure Socket Layer (SSL) setup can also facilitate phishing and man-in-the-middle (MITM) attacks.



M3: Insecure Communication

■ Real-World Case:

- Rapid7 security company disclosed several vulnerabilities attached with kids' smartwatches. Those watches were marketed as ones used by parents for tracking their children and sending them messages or making calls on their smartwatch.
- The watches were supposed to be contacted by approved contact numbers through the mode of a whitelist, but the company found that the **filters were not even working**. The watches even accepted **configuration commands via text messages**. It meant that a hacker could change the watch settings and put children at risk.
- “You can identify where the phone or the child is, you can gain access to audio, or make phone calls to children,” said Deral Heiland, the IoT research lead at Rapid7.



M3: Insecure Communication

Communication Flaws

Insecure communication allows attackers to intercept data as it traverses the Internet.

To ensure secure communication:

- Use SSL/TLS certificates.
- Avoid untrusted and unsigned certificates.
- Use strong encryption protocols.
- Send sensitive data to the back end.
- Avoid sending session IDs with SSL tokens.
- Protect transmission channels with SSL/TLS.



M4: Insecure Authentication

- This category captures notions of authenticating the end user or bad session management such as
 - Failing to identify the user when it is required
 - Failure to maintain the user's identity when it is required.
 - Weaknesses in session management.



M4: Insecure Authentication

■ Real-World Case:

- In 2019, a US bank was hacked by a cyber attacker who took advantage of the bank's website flaw and circumvented the **two-factor authentication** that was implemented for protecting accounts.
- The attacker logged into the system through stolen victim credentials and upon reaching the page where PIN or security answer had to be entered, the attacker used a manipulated string in the Web URL, which had set the computer as a recognized one. This enabled him to cross the stage and initiate the wire transfers.



M4: Insecure Authentication

Authentication Failures

Mobile devices often allow users to log in with default credentials when they fail to recognize users correctly, allowing malicious actors to bypass authentication measures.

To ensure secure authentication:

- Use the right authentication mechanisms and security protocols.
- Deploy authentication on the server-side.
- Avoid storing credentials on mobile devices.
- Encrypt local storage with strong keys.
- Avoid “remember me” options.
- Provide warnings when users opt for “remember me” options.
- Use device-based authentication to prevent users from accessing data from other devices.
- Implement binary attack protections.



M5: Insufficient Cryptography

- The code applies cryptography to a sensitive information asset. However, cryptography is insufficient in some ways. This category covers issues in which cryptography is attempted but not performed correctly. This vulnerability will result in the unauthorized retrieval of sensitive information from the mobile device. To exploit this weakness, an adversary must successfully convert encrypted code or sensitive data into its original unencrypted form due to weak encryption algorithms or flaws in the process of encryption.



M5: Insufficient Cryptography

■ Real-World Case:

- An alert from DHS Industrial Control Systems' Cyber Emergency Response Team and the Philips advisory warned users of a possible vulnerability in the Philips HealthSuite Health Android app.
- The issue which was tracked back to inadequate encryption strength, opened the app to hackers who could get access to users' heart rate activity, blood pressure, sleep state, weight and body composition analysis, etc.



M5: Insufficient Cryptography

Cryptography Flaws

Attackers exploit cryptography flaws to access sensitive data. It is usually easy to exploit weak encryption mechanisms on mobile devices.

To ensure sufficient cryptography:

- Don't store data on mobile devices.
- Use robust cryptography algorithms.



M6: Insecure Authorization

- Authentication verifies the identity of a user or service, and authorization determines their access rights.
- This category captures failures in authorization (e.g., authorization decisions on the client side and forced browsing). It is distinct from authentication issues (e.g., device enrolment and user identification).
- When an app does not authenticate users at all in a situation where it should (e.g., granting anonymous access to some resource or service when authenticated and authorized access is required), then it is an authentication failure and not an authorization failure.



M6: Insecure Authorization

■ Real-World Case:

- The Information security specialists at Pen Test Partners hacked Pandora, a smart car alarm system. In theory, the application is used to track a car, cut off the engine if stolen and lock it until police arrive.
- On the other side of the coin, a hacker can hijack the account and get access to all the data and the smart alarm functionalities. Additionally, they could:
 - Track vehicle movements
 - Enable and disable alarm system
 - Lock and unlock car doors
 - Cut the engine
- In the case of Pandora, hackers got access to everything that was talked about inside the car through the anti-theft system's microphone.



M6: Insecure Authorization

Authorization Failures

Attackers exploit insufficient authorization measures and IDOR vulnerabilities to access sensitive data and escalate privileges. If an app's authorization is incomplete, it might grant user permissions without sufficient verification.

To prevent insecure authorization:

- Avoid granting roles and permissions via mobile devices.
- Verify each identity independently through the back end code.



M7: Client Code Quality

- This category covers “Security Decisions via Untrusted Inputs” and is one of the less frequently used categories. It is the catch-all for code-level implementation problems in the mobile client, which are distinct from server-side coding mistakes. It captures buffer overflows, format string vulnerabilities, and various other code-level mistakes where the solution is to rewrite some code that is running on the mobile device. Most exploitations that fall into this category result in foreign code execution or DoS on remote server endpoints (and not the mobile device itself).
- Poor coding practices can result in vulnerable code. The risk is especially high when team members use different coding techniques and fail to collaborate or provide sufficient documentation. Detecting this vulnerability is challenging because hackers must be aware of the poor coding practices.



M7: Client Code Quality

■ Real-World Case:

- WhatsApp last year patched a vulnerability that hackers were taking advantage of for installing surveillance malware called Pegasus Spyware on smartphones. All they had to do was place a WhatsApp audio call on the targeted phone numbers.
- Within a simple few steps, hackers were able to get in the users' devices and access it remotely.



M7: Client Code Quality

Client-Side Code Quality Issues

Poor coding practices can result in poor and inconsistent code quality. While client code issues are harder for attackers to detect, they pose a significant risk if discovered.

To protect client code quality:

- Implement safe and consistent coding practices.
- Use static code analysis.
- Refrain from simple logic.
- Integrate with reliable libraries.
- List all third-party libraries to patch and update components.
- Use automated tools to test memory leaks and buffer overflow.
- Flag permissions on the content provider to block unauthorized access.



M8: Code Tampering

- This category covers binary patching, local resource modification, method hooking, method swizzling, and dynamic memory modification.
- Once an application is delivered to a mobile device, its code and data resources are resident on the device. An attacker can directly modify the code, change the memory contents dynamically, change or replace the system APIs that the application uses, or modify the application's data and resources. Thus, the attacker can directly subvert the intended use of the software for personal or monetary gain.



M8: Code Tampering

- App stores often contain manipulated versions of mobile applications, such as apps with modified binaries, including malicious content or backdoors. Attackers can deliver these counterfeit applications directly to the victim via phishing or publish them on app stores.
- Usually, in this case, an attacker exploits code modification via malicious forms of the apps hosted in the third-party app stores. They might also trick users into installing an application through phishing attacks.



M8: Code Tampering

Tampering Risks

Counterfeit mobile apps with tampered code are often available on app stores. For instance, attackers may modify an application's binary to install a backdoor.

To address code tampering risks:

- Look for test keys, known APKs, and OTA certificates.
- Check if the build is official using the build.prop element.
- Look for SU binaries and try to use direct SU commands. Substitute/Switch User
- Test-run the SU command to check the current user ID.
- Allow the mobile app to identify and react to code integration.
- Use anti-tampering techniques like validation, code hardening, checksum, and digital signatures.



M9: Reverse Engineering

- Attackers can reverse engineer applications and perform code analysis—this is especially dangerous because attackers can inspect and modify the code to inject malicious functionalities. Reverse engineering allows attackers to understand how an application operates, allowing them to recompile it.
- This category includes the analysis of the final core binary to determine its source code, libraries, algorithms, and other assets. Software such as IDA Pro, Hopper, otool, and other binary inspection tools give the attacker insights into the inner workings of the application. Thus, he/she may exploit other nascent vulnerabilities in the application and uncover information about backend servers, cryptographic constants and ciphers, and intellectual property.



M9: Reverse Engineering

■ Real-World Case:

- Pokemon Go recently faced the security breach glances when it was found that users had reverse engineered the app to know the vicinity of the Pokemons and catch them in minutes.



M9: Reverse Engineering

Reverse Engineering Vulnerabilities

Attackers often reverse engineer and decompile applications to analyze code and insert malicious functionality. Reverse engineering allows attackers to understand how an app works and exploit its weaknesses.

To minimize reverse engineering:

- Check if an attacker can decompile the application using debugging tools.
- Use robust obfuscation, including for metadata.
- Build the app with C or C++ to secure runtime changes.
- Segment the source code to control the app's code flow.
- Use binary packaging to prevent decompilation.
- Implement anti-debugging mechanisms.



M10: Extraneous Functionality

- Often, developers include **hidden backdoor** functionality or other internal development security controls that are not intended to be released into a production environment. **For example, a developer may accidentally include a password as a comment in a hybrid app. Another example involves the disabling of two-factor authentication during testing.**
- Typically, an attacker seeks to understand extraneous functionality within a mobile app to discover hidden functionality in the backend systems. Attackers will typically exploit such extraneous functionality directly from their own systems without any involvement by the end users.



M10: Extraneous Functionality

■ Real-World Case:

- The idea of Wifi File Transfer app was to open port on Android and allow connections from the computer. The problem? An absence of authentication such as passwords, meaning, anyone could connect to a device and get its full access.



M10: Extraneous Functionality

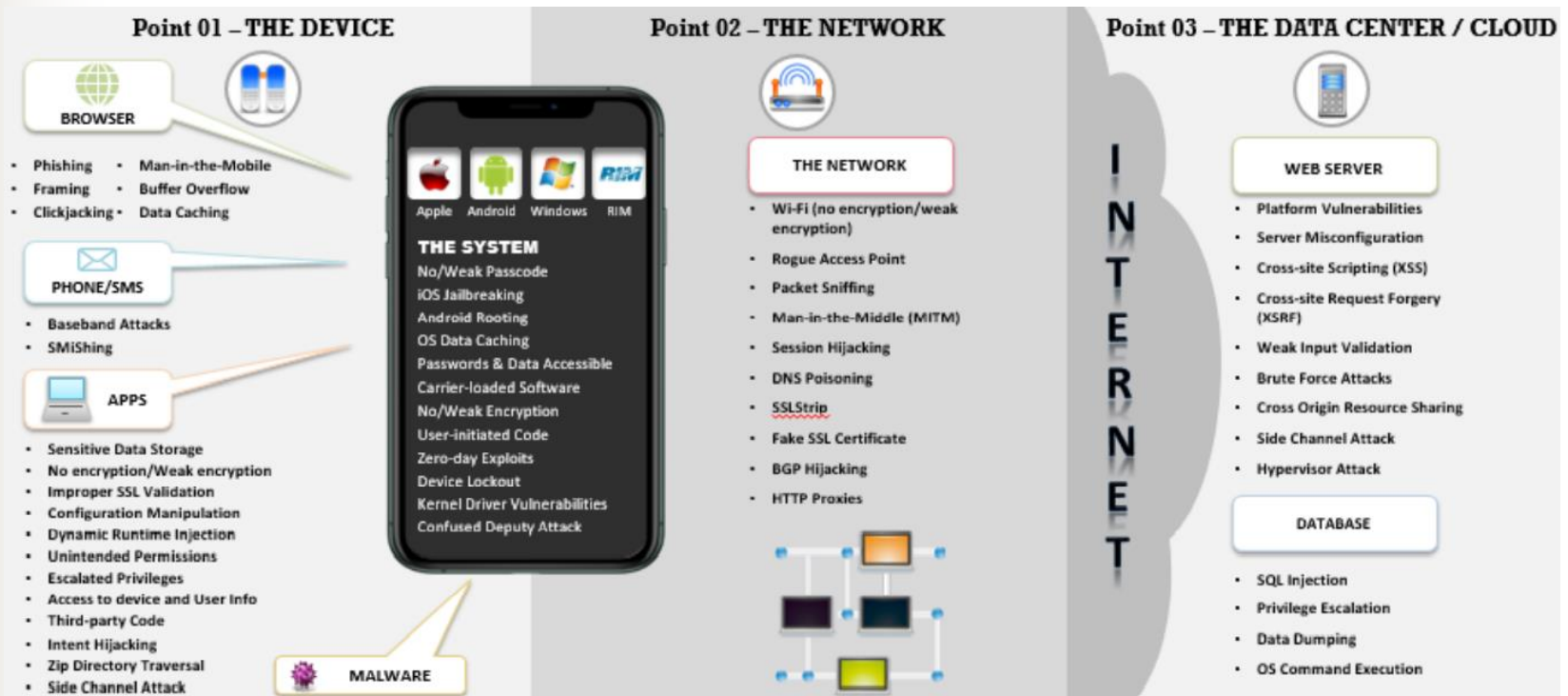
Redundant Functionality

Hackers can exploit many applications with redundant functionality to reach the back end. Some functions allow users to execute a privileged action anonymously.

To address this vulnerability:

- Manually review the code before releasing it to production.
- Analyze API endpoints and check logs and files to verify that attackers cannot exploit hidden functionality.
- Check app configurations for hidden switches.
- Verify that API endpoints and log statements are not publicly available.
- Check if the log contains information about privileged accounts or back end processes.

Anatomy of a Mobile Attack





Point 01 - The Device

■ The Device

■ Browsing-based

- Phishing: Phishing emails or pop-ups redirect users to fake web pages that mimic trustworthy sites, asking them to submit their personal information such as username, password, credit card details, address, and mobile number.
- Framing: Framing involves a web page integrated into another web page using the iFrame elements of HTML.
- Clickjacking: user interface redress attack, is a malicious technique used to trick web users into clicking something different from what they think they are clicking.



Point 01 - The Device

■ The Device

■ Browsing-based

- Man-in-the-Mobile: An attacker implants malicious code into the victim's mobile device to bypass password verification systems that send one-time passwords (OTPs) via SMS or voice calls.
- Buffer Overflow: Buffer overflow is an abnormality whereby a program, while writing data to a buffer, surfeits the intended limit and overwrites the adjacent memory.
- Data Caching: Data caches in mobile devices store information that is often required by these devices to interact with web applications, thereby preserving scarce resources and resulting in better responses time for client applications.



Point 01 - The Device

- Phone/SMS-based
 - Baseband: Attackers exploit vulnerabilities in a phone's GSM/3GPP baseband processor, which sends and receives radio signals to cell towers.
 - Smishing: Use SMS with links or phone number to reveal victim's personal information.



Point 01 - The Device

■ Application-based

- Sensitive Data Storage: Weak security in the database architecture of some apps are vulnerable to exploitation.
- No Encryption/Weak Encryption: Session hijacking attacks common over weak or no encrypted data transmitted.
- Improper SSL Validation:
- Configuration Manipulation: Apps using external configuration files and libraries.
- Dynamic Runtime Injection:
- Unintended Permissions: Misconfigured apps
- Escalated Privileges: Design flaws, programming errors, bugs or configuration oversights.



Point 01 - The Device

■ The Device

■ The System

- **No Passcode/Weak Passcode:**
- **iOS Jailbreaking:** Removing the security mechanisms set by Apple to prevent malicious code from running.
- **Android Rooting:** Gain privileged control within Android subsystem.
- **OS Data Caching:** Extracting sensitive data from OS cache temporarily stored in hard disk by rebooting the victim's device with malicious code.
- **Passwords and Data Accessible:** Exploit vulnerabilities of cryptographic algorithms to decrypt device's keychain exposing user passwords, encryption keys and other private data.
- **Carrier-loaded Software:** Pre-installed software or apps vulnerability exploited,
- **User-initiated Code:** Activity that tricks the victim into installing or clicking links allowing exploiting user browser, cookies and security permissions.



Point 02 - The Network

■ The Network

- **Wi-Fi (weak encryption/no encryption):** weak or no encrypted data is vulnerable to be intercepted by eavesdropping on a wireless connection.
- **Rogue Access Points:** Attackers install an illicit wireless access point which hijacks the connections.
- **Packet Sniffing:** Analyse data packets in network traffic.
- **Man-in-the-Middle (MITM):** Eavesdrop on existing network connection between two systems.
- **Session Hijacking:** Steal valid session IDs.
- **DNS Poisoning:** Exploit network DNS Servers and the substitution of false IP addresses at the DNS level.
- **SSLStrip:** Attackers invisibly downgrades connections from HTTPS to HTTP without encryption.
- **Fake SSL Certificates:** Attacker issues a fake SSL Certificate to intercept on a supposedly secure HTTPS connection.



Point 03 - The Data Centre/CLOUD

■ The Data Centre/CLOUD

■ Web-server-based

- **Platform Vulnerabilities:** Exploit OS, Server software as IIS, application modules running on the webs server.
- **Server Misconfiguration:** Misconfigured web server.
- **Cross-site Scripting (XSS):** Inject malicious script into invalidated input data which are included in dynamic content of server to user's web browser for rendering.
- **Cross-Site Request Forgery (CSRF):** The victim holds an active session with a trusted site and simultaneously visits a malicious site that injects an HTTP request for the trusted site into his/her session, compromising its integrity.
- **Weak Input Validation:** exploit missing validation logic on server to perform unauthorised actions.
- **Brute-Force Attacks:** trial and error approach to guess valid input if number of attempts are not restricted.

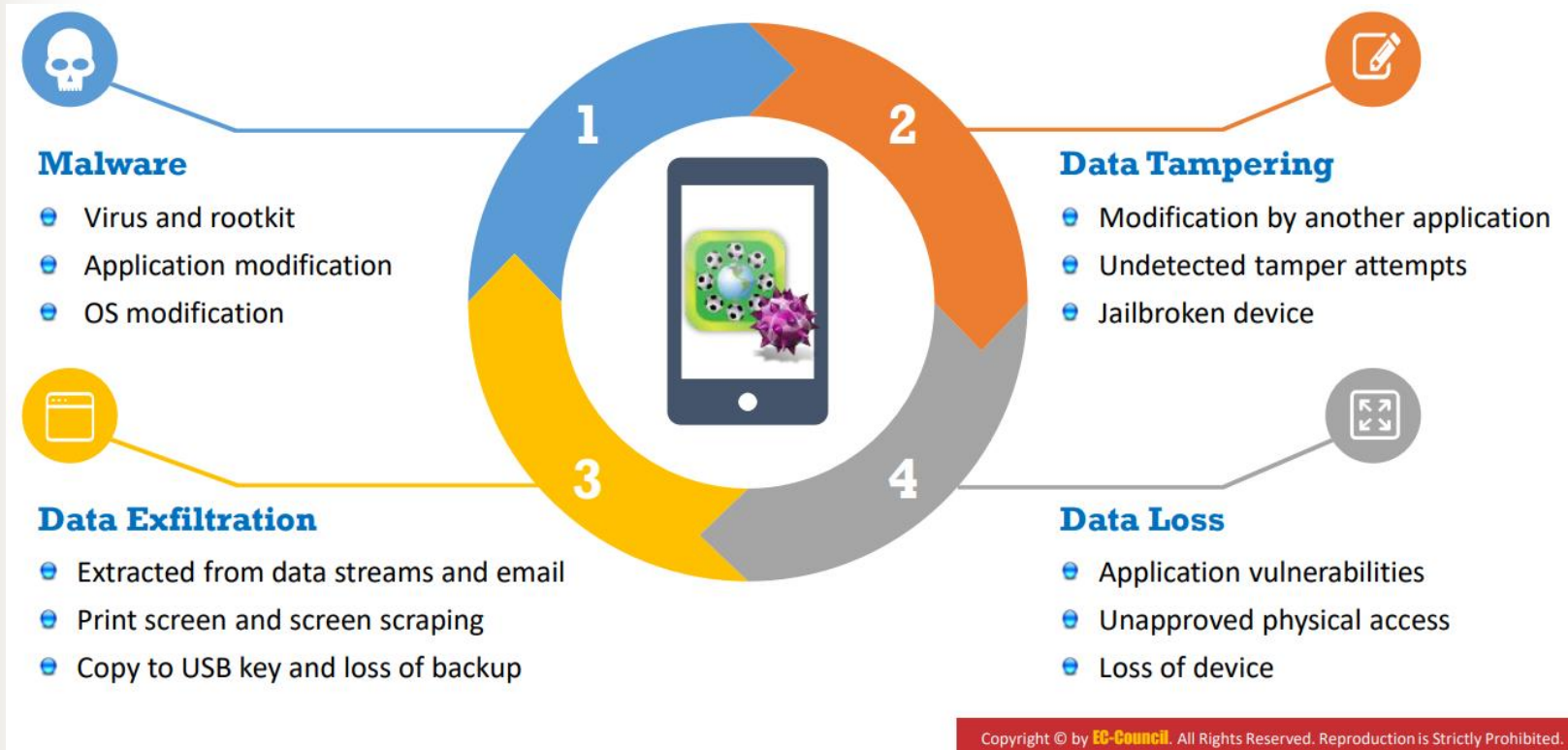


Point 03 - The Data Centre/CLOUD

■ Database Attacks

- **SQL Injection:** Passing SQL commands to be executed on backend database for unauthorised access or retrieve information from database.
- **Privilege Escalation:** Gain high-level access resulting in theft of sensitive data stored in database.
- **Data Dumping:** Cause the database to dump data uncovering sensitive records.
- **OS Command Execution:** Gain unrestricted/root-level system access.

Mobile Attack Vectors





Mobile Platform Vulnerabilities and Risks

- Mobile devices are becoming privileged targets for cyber criminals because of significant improvements in both mobile OS and hardware. In addition, the enhancements in smartphone features introduce new types of security concerns.
- As smartphones are surpassing PCs as preferred devices to access the Internet, manage communications, and so on, attackers are more attracted toward mobile research and implement possible attack schemes against mobile platforms to compromise users' security and privacy or even gain complete control over the victims' devices.

Mobile Platform Vulnerabilities and Risks

- 1 Malicious Apps in Stores
- 2 Mobile Malware
- 3 App Sandboxing Vulnerabilities
- 4 Weak Device and App Encryption
- 5 OS and App Update Issues
- 6 Jailbreaking and Rooting
- 7 Mobile Application Vulnerabilities
- 8 Privacy Issues (Geolocation)
- 9 Weak Data Security
- 10 Excessive Permissions
- 11 Weak Communication Security
- 12 Physical Attacks

Copyright © by **EC-Council**. All Rights Reserved. Reproduction is Strictly Prohibited.

Mobile Attack Countermeasures

OWASP Top 10 Mobile Controls

- ➔ Identify and protect sensitive data on the mobile device
- ➔ Handle password credentials securely on the device
- ➔ Ensure sensitive data are protected in transit
- ➔ Implement user authentication, authorization, and session management correctly
- ➔ Keep the backend APIs (services) and platform (server) secure
- ➔ Secure data integration with third-party services and applications
- ➔ Pay specific attention to the collection and storage of consent for the collection and use of the user's data
- ➔ Implement controls to prevent unauthorized access to paid-for resources
- ➔ Ensure secure distribution /provisioning of mobile applications
- ➔ Carefully check any runtime interpretation of code for errors

<https://www.owasp.org>

Copyright © by IE-Council. All Rights Reserved. Reproduction is Strictly Prohibited.

General Mobile Attack Countermeasures

General Guidelines for Mobile Platform Security

1

Do not load too many **applications** and avoid auto-upload of photos to **social networks**

2

Perform a **Security Assessment** of the Application **Architecture**

3

Maintain **configuration control** and **management**

4

Install applications from trusted application **stores**

5

Securely **wipe or delete** the data when disposing of the device

6

Do not share information within **GPS-enabled apps** unless necessary

7

Disable wireless access, such as **Wi-Fi** and **Bluetooth**, if not in use

8

Never connect two separate networks, such as **Wi-Fi** and **Bluetooth**, simultaneously

Copyright © by **EG-Council**. All Rights Reserved. Reproduction Is Strictly Prohibited.



General Guidelines

- Given below are various guidelines that help you to protect your mobile device:
- Do not load too many applications and avoid auto-upload of photos to social networks
- Perform a security assessment of the application architecture
- Maintain configuration control and management
- Install applications from trusted application stores
- Securely wipe or delete the data when disposing of the device
- Do not share the information within GPS-enabled apps unless it is necessary
- Never connect two separate networks such as Wi-Fi and Bluetooth simultaneously



General Guidelines

- Disable wireless access such as Wi-Fi and Bluetooth if not in use
 - Ensure that your Bluetooth is “off” by default. Turn it on whenever it is necessary
 - Disable wireless access such as Wi-Fi and Bluetooth if not in use to avoid illegal wireless access to the device
 - Disable sharing/tethering Internet connections over Wi-Fi and Bluetooth when not in use



General Guidelines

■ Use Passcode

- Configure a strong passcode with the maximum possible length to gain access to your mobile devices
- Set an idle timeout to automatically lock the phone when not in use
- Enable the lockout/wipe feature after a certain number of attempts
- Consider an eight-character complex passcode
- Thwart passcode guessing: set erase data to ON



General Guidelines

- Update OS and Apps
 - Update OS and apps to keep them secure
 - Apply software updates when new releases are available
 - Perform regular software maintenance
- Enable Remote Management
 - In an enterprise environment, use MDM software to secure, monitor, manage, and support mobile devices deployed across the organization
- Do not allow Rooting or Jailbreaking
 - Ensure that your MDM solutions prevent or detect rooting/jailbreaking
 - Include this clause in your mobile security policy



General Guidelines

■ Use Remote Wipe Services

- Use remote wipe services such as Find My Device (Android) and Find My iPhone or FindMyPhone (Apple iOS) to locate your device should it be lost or stolen
- Report a lost or stolen device to IT so that they can disable certificates and other access methods associated with the device

■ Encrypt Storage

- If supported, configure your mobile device to encrypt its storage with hardware encryption
- Use device encryption and patch applications
- Encrypt the device and backups



General Guidelines

- Perform periodic backup and synchronization
 - Use a secure, over-the-air backup-and-restore tool that performs periodic background synchronization
 - (Android) Backup to your Google account so that sensitive enterprise data are not backed up to the cloud
 - Control the location of backups
 - Encrypt backups
 - Keep sensitive data off shared mobile devices. If enterprise information is locally stored on a device, then it is recommended that this device not be openly shared
 - Limit logging data stored on the device
 - Use a secure data-transfer utility or encrypt data in transit to or from the device, to ensure confidentiality and data integrity



Security Issues arising from App Stores

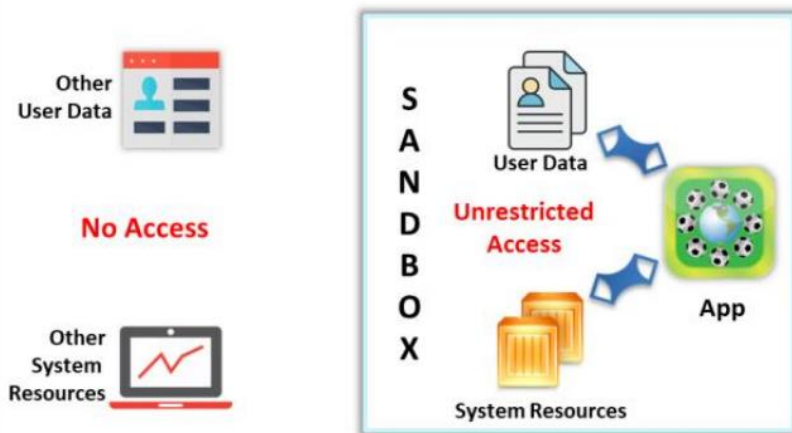
- App sandboxing is a security mechanism that helps protect systems and users by limiting the resources that an app can access to its intended functionality on the mobile platform.
- Often, sandboxing is useful in executing untested code or untrusted programs from unverified or untrusted third parties, suppliers, users, and websites.
- This enhances security by isolating the app to prevent intruders, system resources, malware such as Trojans and viruses, and other apps from interacting with it.
- As sandboxing isolates applications from one another, it protects them from tampering with each other; however, malicious applications may exploit vulnerabilities and bypass the sandbox



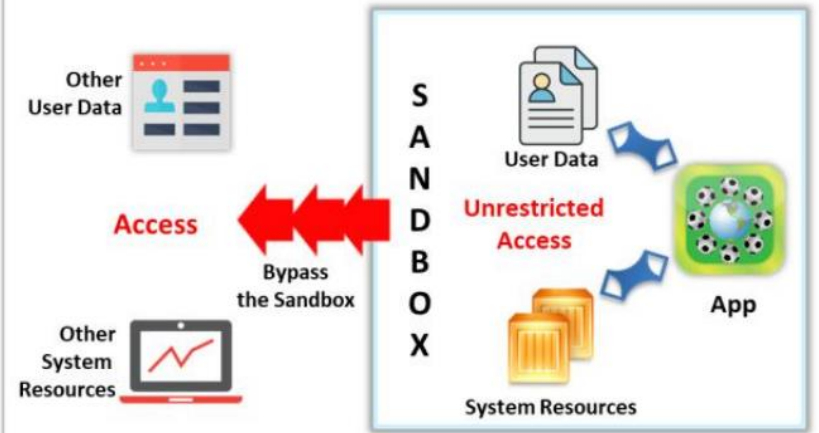
App Sandboxing Issues

- A secure sandbox environment provides an application with limited privileges intended for its functionality to restrict it from accessing other users' data and system resources, whereas a vulnerable sandbox environment allows a malicious application to exploit vulnerabilities in the sandbox and breach its perimeter, resulting in the exploitation of other data and system resources.

Secure Sandbox Environment



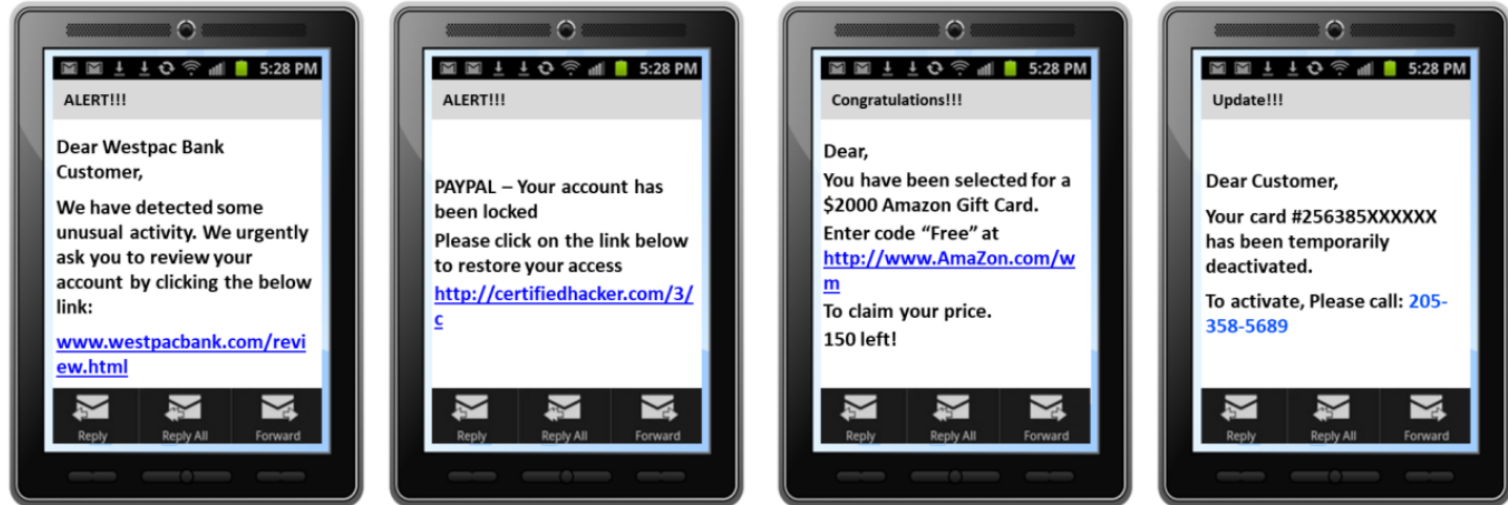
Vulnerable Sandbox Environment



App Sandboxing issues

SMS Phishing

SMS Phishing Attack Examples



SMS Phishing is the act of trying to acquire personal and financial information by sending SMSs(Instant Messages or IMs) containing deceptive links



SMiShing

- SMS phishing (also known as SMiShing) is a type of phishing fraud in which an attacker uses SMS systems to send bogus text messages. It is the act of trying to acquire personal and financial information by sending SMS (or IM) containing deceptive links. Often, these bogus text messages contain a deceptive website URL or telephone number to lure victims into revealing their personal or financial information, such as SSNs, credit card numbers, and online banking username and password. In addition, attackers implement SMiShing to infect victims' mobile phones and associated networks with malware



SMiShing

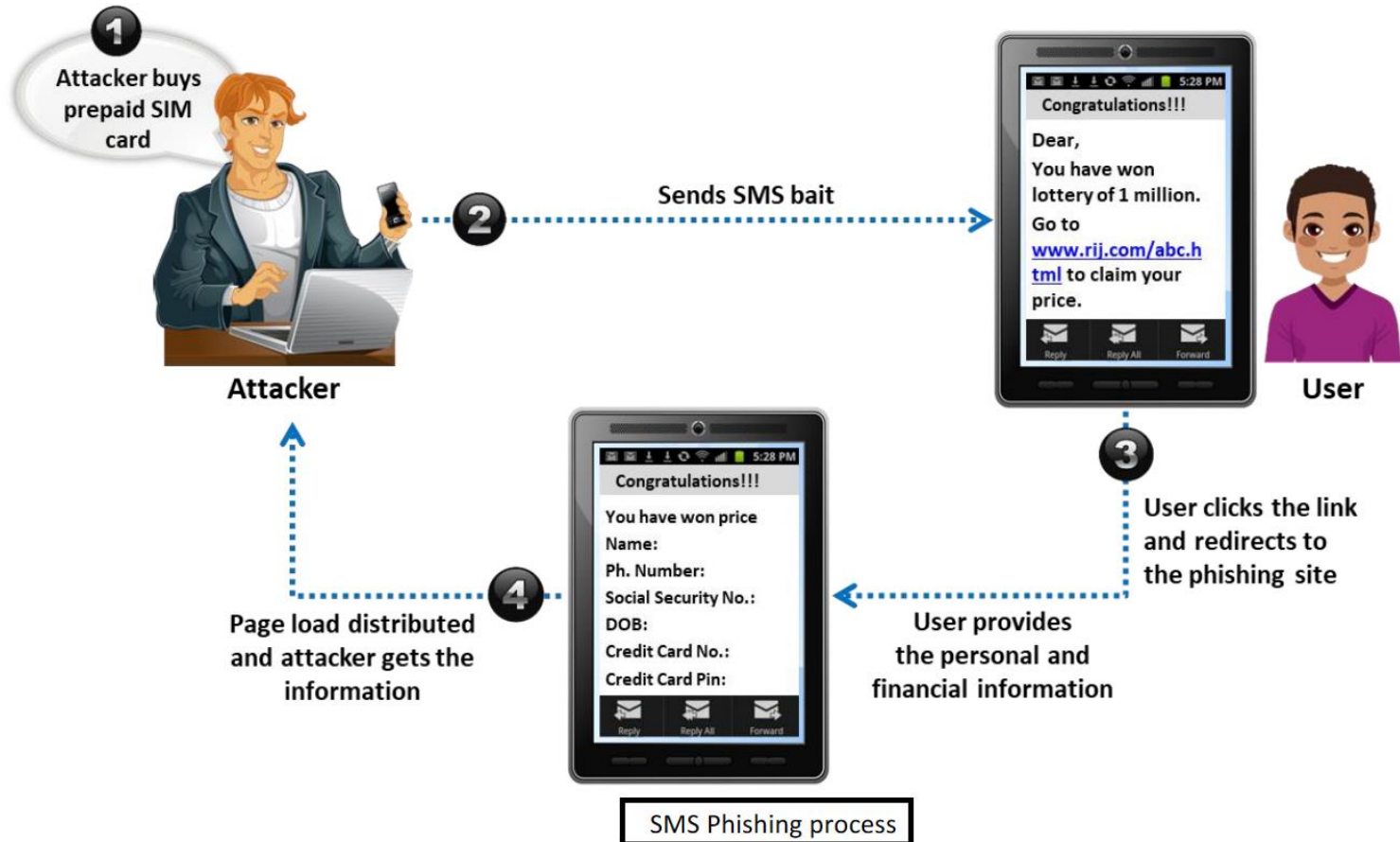
- Attackers buy a prepaid SMS card using a fake identity. Then, they send an SMS bait to a user. The SMS may seem attractive or urgent. For example, it may include a lottery message, gift voucher, online purchase, or notification of account suspension, along with a malicious link or phone number. When the user clicks the link, considering it to be legitimate, he/she is redirected to the attacker's phishing site, where he/she provides the requested information (e.g., name, phone number, date of birth, credit card number or PIN, CVV code, SNN, and email address). The attacker may use the acquired information to perform malicious activities such as identity theft, online purchases, and so on.



Why SMiShing is effective?

- Most consumers access the Internet through a mobile device.
- Easy to set up a mobile phishing campaign.
- Difficult to detect and stop it causes harm.
- Mobile users are not conditioned to receiving spam text messages on their mobile devices.
- No mainstream mechanism for weeding out spam SMS.
- Most mobile anti-virus tools do not check SMS.

Mobile Attacks and Countermeasures





Mobile Spam

- Also known as **SMS spam, text spam, or m-spam**, refers to unsolicited messages sent in bulk form to known/unknown phone numbers/email IDs to target mobile phones.
- Spam is a generic term for unsolicited messages sent via electronic communication technologies such as **SMS, MMS, instant messaging (IM), and email**.
- Spam messages consume a significant amount of network bandwidth. The consequences of mobile spam include financial loss, malware injection, and corporate data breach incidents.



Mobile Spam

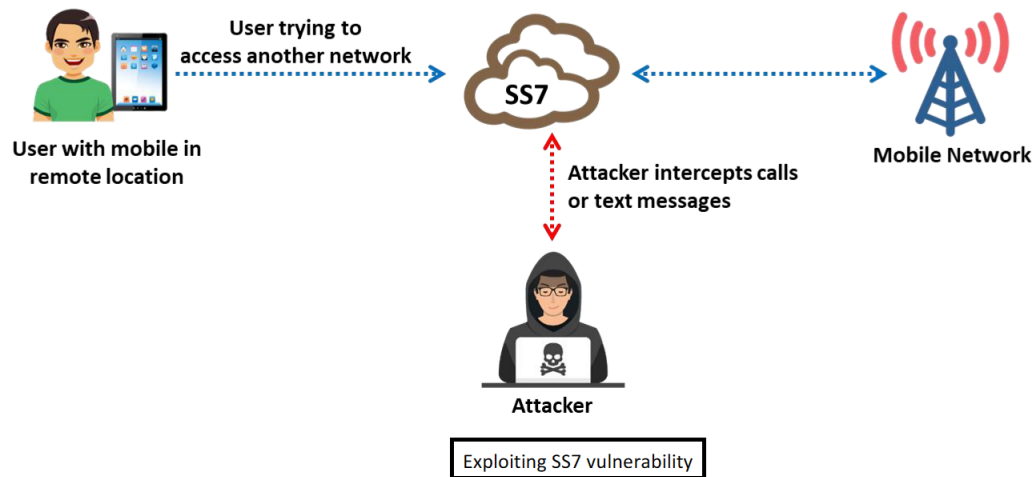
- Typical spam messages delivered to mobile phones are as follows:
 - Messages containing advertisements or malicious links that can trick users into revealing confidential information
 - Attractive commercial messages advertising products/services
 - SMS or MMS messages claiming that the victim has won a prize and asking him/her to place a call to a provided premium-rate telephone service number for further details
 - Malicious links that may lure users into divulging sensitive personal or corporate data
 - Phishing messages that lure the recipient into revealing personal or financial data such as name, address, date of birth, bank account number, credit card number, and so on, which an attacker can use to commit identity or financial fraud



Example of a spam message

Signaling System 7 (SS7) is a communication protocol that allows mobile users to exchange communication through another cellular network (especially when roaming).

Ethical Hacking Essentials
Mobile Attacks and Countermeasures



Changing the telecom operator or using the network of another cell tower is allowed via the SS7 protocol. It is operated depending on mutual trust between the operators, without any authentication verification.



Exploiting SS7 Vulnerability

- Signaling System 7 (SS7) is a communication protocol that allows mobile users to exchange communication through another cellular network (especially when roaming).
- Mobile devices are meant to be carried across different locations to serve their users.
- Changing the telecom operator or using the network of another cell tower is allowed via the SS7 protocol.
- This signaling mechanism is operated depending on mutual trust between the operators, without any authentication verification. Since the SS7 signaling network is not isolated, the attacker can exploit this vulnerability to perform an MITM attack by impeding text messages and calls between the communicating devices.
- The attacker can eavesdrop on bank credentials, OTPs and other sensitive information routed through the network.
- This vulnerability in SS7 can also allow the attacker to bypass two-factor authentication and end-to-end encryption via SMS.



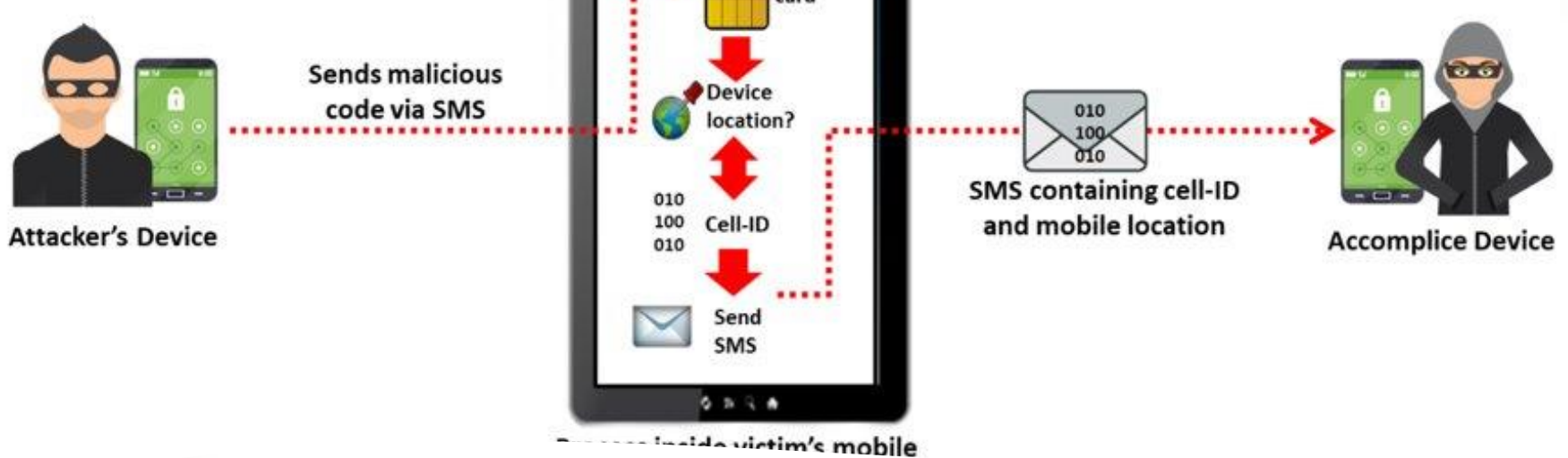
Threats Associated with SS7 vulnerability

- When the attacker gains access to the SS7 protocol, the victim's device faces the following risks:
 - Exposing the subscriber's identity
 - Revealing the network identity
 - Spying on and intercepting the network to steal personal data
 - Allowing phone tapping
 - Performing DoS attacks to damage the reputation of the target telecom operator
 - Tracking geographic locations
- All Rights Reserved.
Reproduction is Strictly Prohibited.



Simjacker

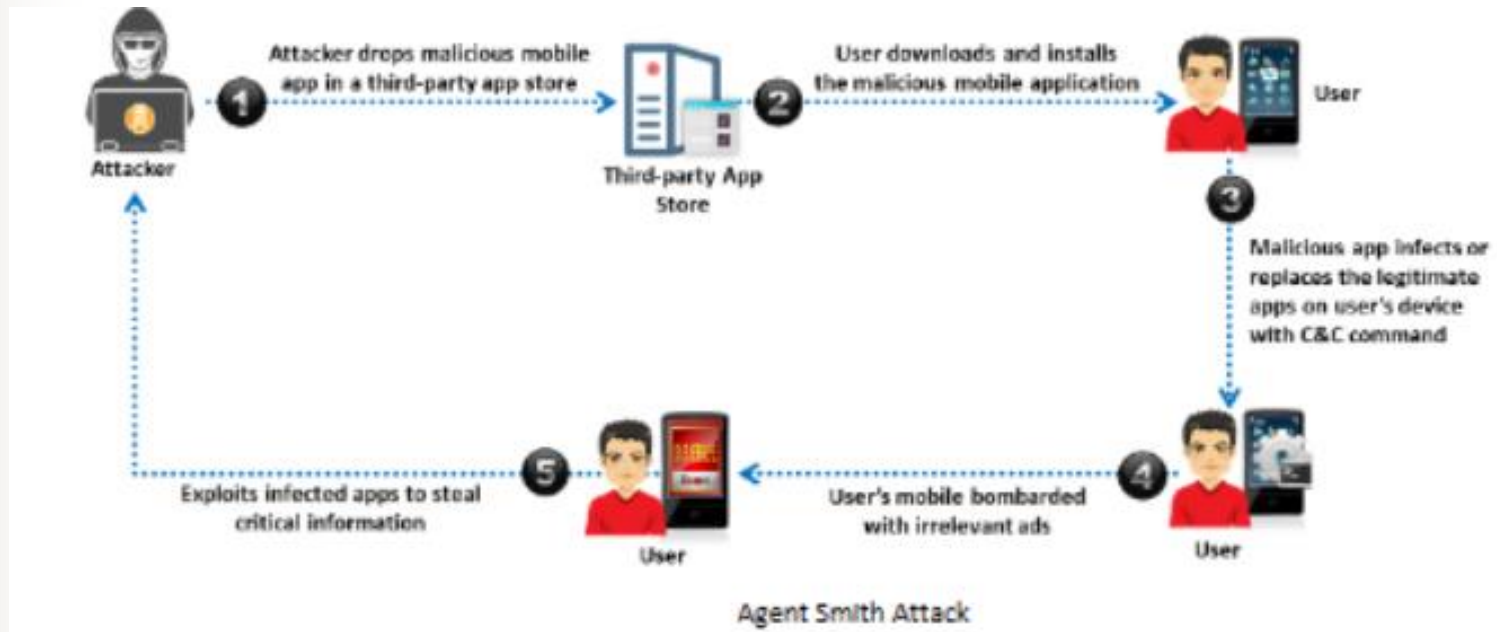
- Simjacker is a vulnerability associated with a SIM card's S@T browser (SIMalliance Toolbox Browser), a pre-installed software incorporated in SIM cards to provide a set of instructions.
- Attackers exploit this vulnerability in the S@T browser to perform various malicious activities such as capturing the device location, monitoring calls, gathering information such as IMEI, making fraudulent or expensive calls, sending premium-rate messages, forcing the device browser to connect to malicious websites, and performing DoS attacks to block SIM cards.
- The SIM card-based attack can be aggravated based on the victim's device. The Simjacker attack is initiated by sending spyware-like code in the form of system or SIM card settings through an SMS to take complete control of the SIM card and mobile device to issue various commands without user interaction



Steps involved in Simjacker attack

- The attacker sends fraudulent SMS containing hidden code or instructions from a SIM Application Toolkit (STK)
- The victim receives the malicious SMS and the S@T browser on the SIM card automatically recognizes and processes the hidden instructions or code
- The injected code performs various activities on the device without the user's consent
- The accomplice device receives the user information via SMS, which an attacker can use to track live locations, exfiltrate the device information, and perform many other malicious activities

Agent Smith Attack



- An Agent smith attack is carried out by persuading the victim to install a malicious app designed and published by an attacker.
- The malicious app replaces legitimate apps, such as WhatsApp, SHAREit, and MX Player.
- The attacker produces a huge volume of advertisements on the victim's device through the infected app for financial gains

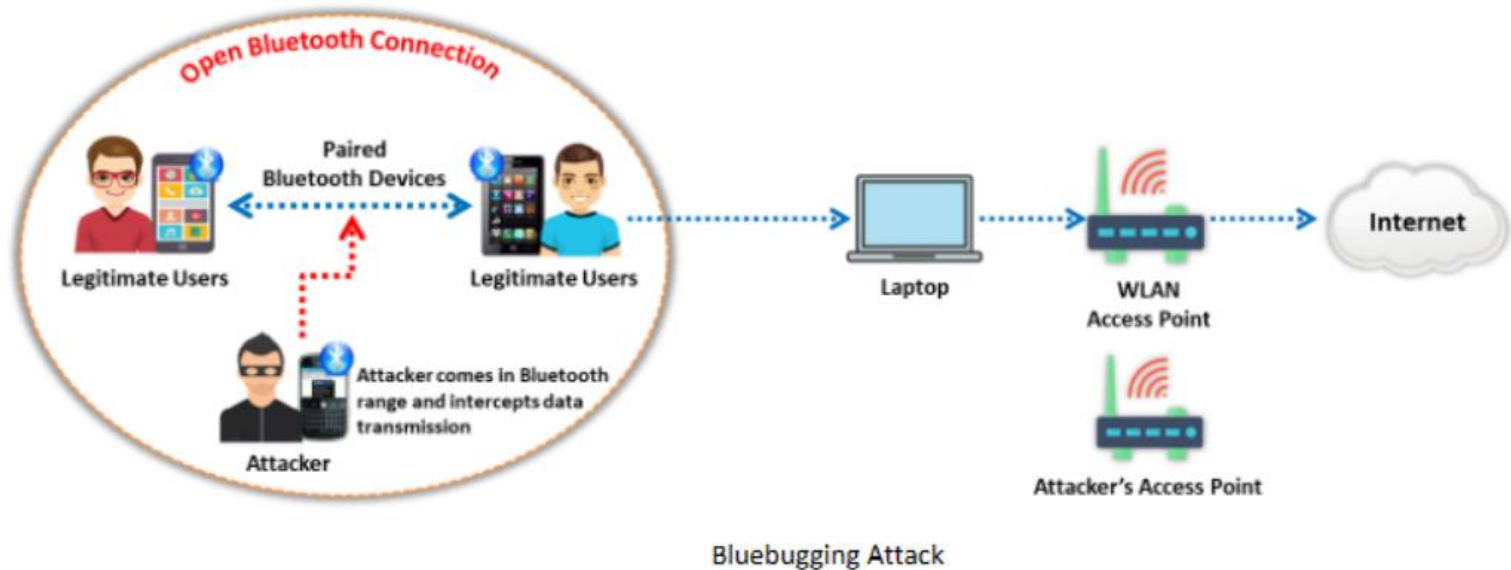


Agent Smith Attack

- Agent Smith attacks are carried out by luring victims into downloading and installing malicious apps designed and published by attackers in the form of games, photo editors, or other attractive tools from third-party app stores such as 9Apps.
- Once the user has installed the app, the core malicious code inside the application infects or replaces the legitimate apps in the victim's mobile device C&C commands.
- The deceptive application replaces legitimate apps such as WhatsApp, SHAREit, and MX Player with similar infected versions.
- The application sometimes also appears to be an authentic Google product such as Google Updater or Themes.
- The attacker then produces a massive volume of irrelevant and fraudulent advertisements on the victim's device through the infected app for financial gain. Attackers exploit these apps to steal critical information such as personal information, credentials, and bank details, from the victim's mobile device through C&C commands.

Mobile device pairing on open connections (public Wi-Fi/unencrypted Wi-Fi routers)

- Allows attackers to eavesdrop and intercept data transmission using techniques such as;
 - Bluesnarfing (stealing information via Bluetooth)
 - Bluebugging (gaining control over the device via Bluetooth)





Mobile device pairing on open connections (public Wi-Fi/unencrypted Wi-Fi routers)

- Techniques such as “bluesnarfing” and “bluebugging” help an attacker to eavesdrop on or intercept data transmission between mobile devices paired on open connections (e.g., public Wi-Fi or unencrypted Wi-Fi routers).



Bluesnarfing

- Bluesnarfing (Stealing information via Bluetooth)
- Bluesnarfing is the theft of information from a wireless device through a Bluetooth connection, often between phones, desktops, laptops, PDAs, and other devices.
- This technique allows an attacker to access the victim's contact list, emails, text messages, photos, videos, and business data, stored on the device.
- Any device with its Bluetooth connection enabled and set to “discoverable” (allowing other Bluetooth devices within range to view the device) may be susceptible to bluesnarfing if the vendor's software contains a certain vulnerability.
- Bluesnarfing exploits others' Bluetooth connections without their knowledge



Bluebugging

- Bluebugging (Taking over a device via Bluetooth)
- Bluebugging involves gaining remote access to a target Bluetooth-enabled device and using its features without the victim's knowledge or consent.
- Attackers compromise the target device's security to perform a backdoor attack prior to returning control to its owner.
- Bluebugging allows attackers to sniff sensitive corporate or personal data, receive calls and text messages intended for the victim, intercept phone calls and messages, forward calls and messages, connect to the Internet, and perform other malicious activities such as accessing contact lists, photos, and videos.



Mobile Device Management (MDM)

- MDM provides platforms for over-the-air or wired distribution of applications, data, and configuration settings for all types of mobile devices, including mobile phones, smartphones, tablet computers, and so on.
- It helps in implementing enterprise-wide policies to reduce support costs, business discontinuity, and security risks.
- It helps system administrators to deploy and manage software applications across all enterprise mobile devices to secure, monitor, manage, and support these devices.
- It can be used to manage both company-owned and employee-owned (BYOD) devices across the enterprise.
- Examples of MDM solutions include IBM MaaS360, Citrix Endpoint Management, VMware AirWatch, and so on

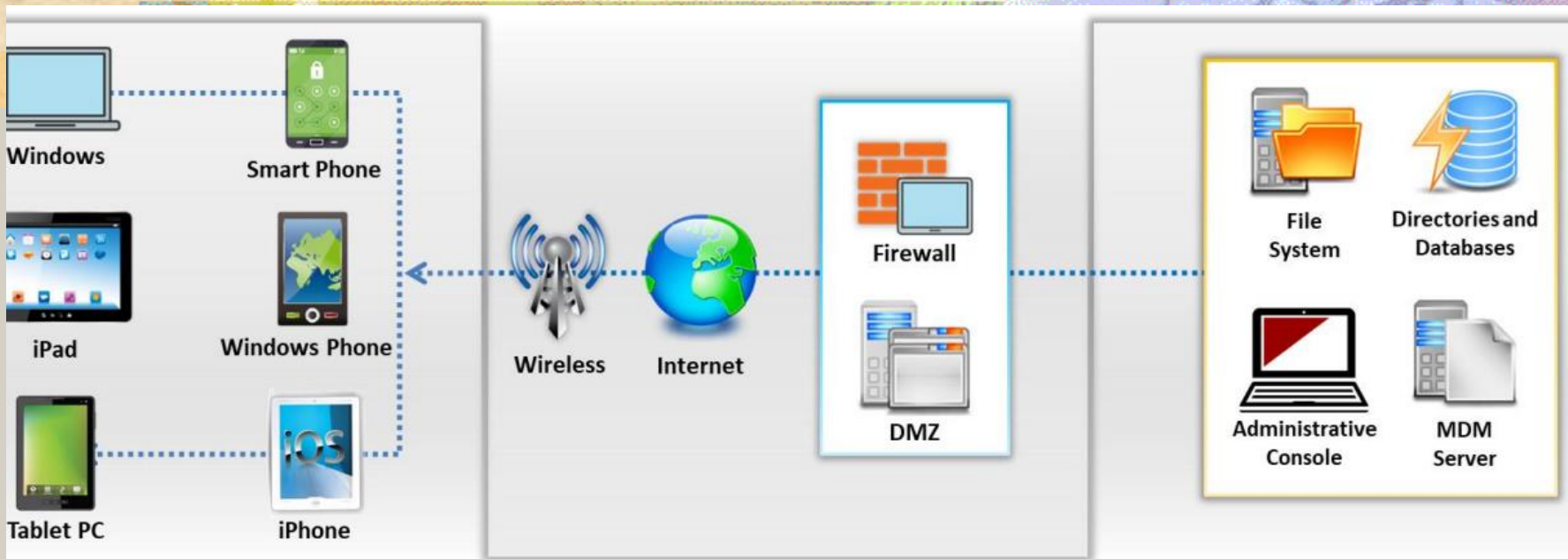


Figure 1. MDM Architecture

MDM

- The basic features of MDM software are as follows:
 - Uses a passcode for the device
 - Remotely locks the device if it is lost
 - Remotely wipes data in the lost or stolen device
 - Detects if the device is rooted or jailbroken
 - Enforces policies and tracks inventory
 - Performs real-time monitoring and reporting



BYOD

- BYOD refers to a policy that allows employees to bring their personal devices such as laptops, smartphones, and tablets to their workplace and use them for accessing the organization's resources as per their access privileges.
- BYOD allows employees to use devices that they are comfortable with and which best fits their preferences and work purposes.
- With a “work anywhere, anytime” strategy, the challenge for the BYOD trend is to secure the company's data and meet compliance requirements.



BYOD Benefits

- **Increased Productivity:** Employees become experts in using their personal devices and this increases their productivity. In addition, users tend to upgrade their personal devices with cutting-edge technologies so that the enterprise can benefit from the latest features (both software and hardware) of the device.
- **Employee Satisfaction:** By implementing BYOD, employees use devices of their own choice, which they invest in themselves without the company's involvement. Moreover, employees are more comfortable with their personal devices, as they contain both personal data and corporate data, thus eliminating the usage of multiple devices



BYOD Benefits

- **Work Flexibility:** By practicing BYOD, employees can carry a single device to satisfy their personal and professional needs. Work that is usually done in the office can be done from anywhere in the world, as employees are provided with access to the corporate data. BYOD users have more freedom, as their companies do not impose strict rules that they would have to follow when using company property. BYOD replaces the traditional client-server model with a mobile and cloud-centric strategy, which can have far reaching benefits.
- **Lower Costs:** A business that adopts BYOD does not have to spend on devices but saves money, as employees purchase their own devices. In addition, the cost of data services shifts to employees who can take better care of their own property (device).

BYOD Risks

BYOD Risks

01

Sharing **confidential data**
on unsecured networks

02

Data leakage and
endpoint security issues

03

Improperly **disposing**
of devices

04

Support for many
different devices

05

Mixing personal
and **private data**

06

Lost or **stolen**
devices

07

Lack of awareness

08

Ability to bypass
organization's
network policies



BYOD Risks

- **Sharing confidential data on unsecured networks:** Employees might access corporate data via a public network. These connections may not be encrypted; sharing confidential data via an unsecured network may lead to data leakage.
- **Data leakage and endpoint security issues:** In this cloud-computing era, mobile devices are insecure endpoints with cloud connectivity. By synchronizing with organizational email or other apps, these mobile devices carry confidential information. If the device is lost, it could potentially expose all the corporate data.
- **Improperly disposing of devices:** An improperly disposed of device could contain a wealth of sensitive information, such as financial information, credit card details, contact numbers, and corporate data. Therefore, it is important to ensure that the device does not contain any data before it is disposed of or passed on to others.



BYOD Risks

- **Support for many different devices:** Organizations allow employees to access its resources from anywhere in the world, enhancing productivity and driving employee satisfaction. However, support for different devices and processes can increase costs. Employee-owned devices have limited security and come with a variety of platforms. This impedes the IT department's capability to manage and control all the devices in a company.
- **Mixing personal and private data:** Mixing personal and corporate data on mobile devices leads to serious security and privacy implications. Therefore, it is a good practice to keep the corporate data separate from the employee's personal data; this helps an organization to apply specific security measures such as encryption to protect the critical corporate data stored on the mobile device. In addition, it becomes easy for the organization to remotely wipe the corporate data without affecting the employee's personal data when an employee leaves the organization.



BYOD Risks

- **Lost or stolen devices:** Due to their small size, mobile devices are often lost or stolen. When an employee loses his/her mobile device that is used for both personal and official purposes, the organization might face a security risk, as attackers can compromise the corporate data stored in the lost device.
- **Lack of awareness:** Organizations must educate their employees regarding BYOD security issues. Failing to do so might result in compromising the corporate data stored in mobile devices.
- **Ability to bypass organization's network policy rules:** According to their particular requirements, the policies imposed may differ between wired networks and wireless networks. BYOD devices connected to wireless networks have the ability to bypass the organization's network policy rules enforced only on wired LANs.



BYOD Risks

- **Infrastructure issues:** A BYOD program involves dealing with various platforms and technologies. Not all employees carry the same devices. Different devices, each running different OS and programs, come with their own security loopholes. Thus, it can be problematic for an IT department to set up and maintain infrastructure to support different devices' needs, such as managing data, security, backup, and compatibility among devices.
- **Disgruntled employees:** Disgruntled employees in an organization can misuse corporate data stored on their mobile devices. They may also leak sensitive information to competitors.